

Operaciones con Matrices aplicando Programación Orientada a Objetos

Proyecto CMatriz — Apache NetBeans IDE

Desarrollo, documentación y evidencias de ejecución de diez métodos para el tratamiento de matrices cuadradas de números enteros, implementados sobre una arquitectura de clases con encapsulamiento, herencia, abstracción y polimorfismo.

8	3	12	5
7	20	1	9
4	15	6	11
2	10	14	13

Asignatura: Algoritmos y Estructura de Datos

Tema: Arreglos bidimensionales (matrices)

Estudiante: Taípe Sulla Fernando

Docente: Fernández Bejarano Raúl Enrique

Ciclo: V ciclo

Índice de contenidos

1. Introducción
2. Objetivos del trabajo
3. Marco teórico
4. Estructura del proyecto y diseño de clases
5. Metodología y matriz de prueba
6. Actividad 1: Eliminar un elemento "x" de la matriz
7. Actividad 2: Modificar un elemento "x" de la matriz
8. Actividad 3: Sumar los elementos de la matriz
9. Actividad 4: Número menor, número mayor y promedio
10. Actividad 5: Suma de los contenidos extremos que forman la letra L
11. Actividad 6: Suma de los contenidos extremos que forman la letra J
12. Actividad 7: Suma de los contenidos extremos que forman la letra U
13. Actividad 8: Menor número de cada fila
14. Actividad 9: Contar los números pares de la matriz
15. Actividad 10: Mover las filas una posición a la derecha
16. Resumen general de resultados
17. Conclusiones
18. Recomendaciones
19. Anexo A: guía de ejecución en NetBeans
20. Anexo B: código fuente completo

Contenido del entregable

Archivo	Descripción
Informe_Matrices_P00.pdf	Este documento: fundamentación, código explicado paso a paso y evidencias de ejecución.
CMatriz.zip	Proyecto completo de Apache NetBeans, listo para abrir, compilar y ejecutar.

1. Introducción

Las **matrices** o arreglos bidimensionales son estructuras de datos que permiten almacenar información organizada en filas y columnas bajo un mismo nombre de variable. Su utilidad es enorme en el desarrollo de software: se emplean en el tratamiento de imágenes, en hojas de cálculo, en videojuegos basados en cuadrículas, en sistemas de reservas de asientos y en prácticamente cualquier problema donde los datos tengan una disposición tabular.

El presente informe documenta el desarrollo de **diez métodos** que operan sobre una matriz cuadrada de números enteros, ampliando el proyecto `CMatrix` elaborado previamente. Las operaciones abarcan desde la modificación y eliminación de elementos hasta el recorrido de figuras geométricas (las letras **L**, **J** y **U**) y el desplazamiento circular de filas.

El trabajo no se limita a resolver los ejercicios: toda la solución se construye bajo el paradigma de la **Programación Orientada a Objetos**, separando la estructura de datos (clase abstracta `Matriz`), el contrato de operaciones (interface `OperacionesMatriz`), la lógica del negocio (clase `CMatrix`) y la interacción con el usuario (clase `AplicaMatriz`). Cada método se acompaña de su análisis, su explicación paso a paso, la evidencia de ejecución obtenida en consola y la verificación manual del resultado.

2. Objetivos del trabajo

2.1. Objetivo general

Desarrollar e implementar en el lenguaje Java un conjunto de métodos para el tratamiento de matrices cuadradas, aplicando los principios de la Programación Orientada a Objetos y documentando su funcionamiento mediante evidencias de ejecución verificables.

2.2. Objetivos específicos

- Implementar métodos de mantenimiento de datos que permitan **eliminar** y **modificar** elementos dentro de una matriz.
- Aplicar las técnicas de **acumulador** y **contador** para obtener la suma total, el promedio y la cantidad de números pares de la matriz.
- Emplear la **comparación sucesiva** para determinar el mayor, el menor y el mínimo de cada fila.
- Manejar con precisión los **índices de filas y columnas** para recorrer figuras específicas formadas por los elementos extremos de la matriz (letras **L**, **J** y **U**).
- Resolver el **desplazamiento circular** de los elementos de cada fila sin pérdida de información.
- Demostrar el uso del **encapsulamiento**, **la herencia**, **la abstracción** y **el polimorfismo** en el diseño de las clases.
- Elaborar un documento técnico con evidencias que sirva como material de portafolio profesional.

3. Marco teórico

3.1. Las matrices en Java

Una matriz es un arreglo de dos dimensiones. En Java se declara indicando dos pares de corchetes y se crea reservando memoria con el operador `new`:

```
int[][] matriz;           // declaración de la referencia
matriz = new int[4][4];   // creación: 4 filas por 4 columnas = 16 celdas
matriz[0][2] = 12;        // asignación: fila 1, columna 3 (los índices empiezan en 0)
```

Para acceder a un elemento se utilizan dos índices: el primero identifica la **fila** y el segundo la **columna**. Es indispensable recordar que en Java los índices comienzan en **0**; por ese motivo, en los mensajes dirigidos al usuario se muestra `i + 1` y `j + 1`, que resulta más natural para quien lee la salida.

El recorrido habitual de una matriz se realiza con dos bucles `for` anidados: el externo controla las filas y el interno las columnas. Este recorrido tiene una complejidad de $O(n^2)$, ya que se visitan $n \times n$ elementos. En cambio, los métodos que solo recorren los bordes (letras L, J y U) tienen complejidad $O(n)$, pues únicamente visitan una cantidad proporcional a la dimensión.

3.2. Los pilares de la Programación Orientada a Objetos

Pilar	Definición	Aplicación en este proyecto
Encapsulamiento	Ocultar los datos internos y exponerlos solo mediante métodos controlados.	Los atributos <code>matriz</code> y <code>tamano</code> son <code>private</code> ; se accede a ellos con <code>getElemento()</code> y <code>setElemento()</code> .
Abstracción	Definir qué hace un objeto sin detallar cómo lo hace.	La clase abstracta <code>Matriz</code> declara <code>escritura()</code> sin implementarla, y la interface <code>OperacionesMatriz</code> define el contrato de las diez actividades.
Herencia	Una clase hija reutiliza los atributos y métodos de una clase padre.	<code>CMatrix extends Matriz</code> : hereda los atributos, los constructores y los métodos de acceso sin volver a escribirlos.
Polimorfismo	Un mismo método puede comportarse de distinta forma según el objeto.	Sobrescritura con <code>@Override</code> de <code>escritura()</code> , <code>toString()</code> y <code>equals()</code> ; además de la sobrecarga de constructores.

3.3. Técnicas algorítmicas aplicadas

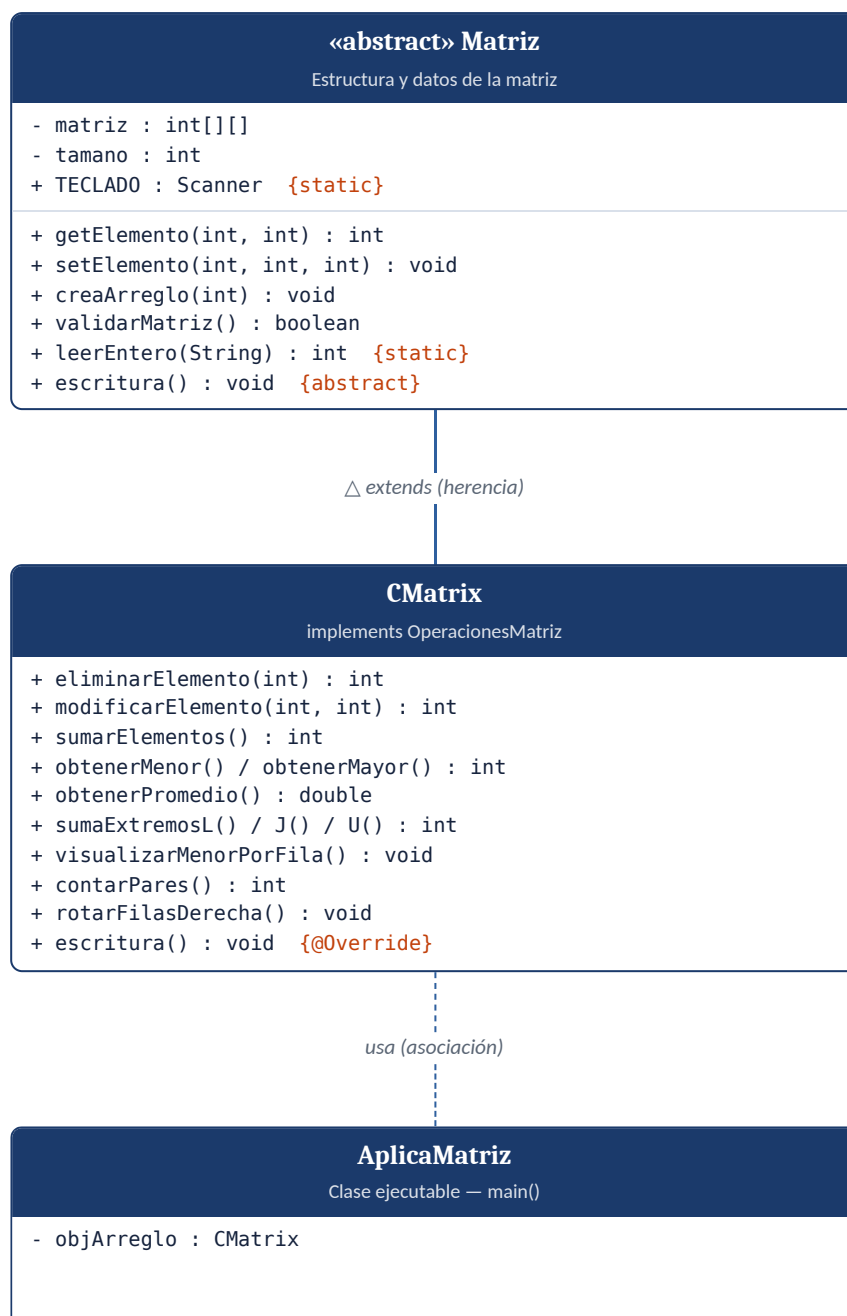
Técnica	En qué consiste	Actividades
Acumulador	Variable iniciada en 0 que va sumando los valores recorridos.	3, 4, 5, 6, 7
Contador	Variable que se incrementa en 1 cada vez que se cumple una condición.	1, 2, 9
Comparación sucesiva	Se guarda un candidato y se reemplaza cuando aparece uno mejor.	4, 8
Búsqueda secuencial	Recorrido completo comparando cada elemento con un valor buscado.	1, 2
Variable auxiliar	Almacenamiento temporal de un dato que se perdería al ser sobrescrito.	10

4. Estructura del proyecto y diseño de clases

El proyecto fue organizado en cuatro archivos dentro del paquete `clases`. Los archivos marcados en color naranja fueron **creados o ampliados** en el desarrollo de estas actividades.

```
CMatriz/
├─ build.xml
├─ manifest.mf
├─ nbproject/           (archivos de configuración de NetBeans)
├─ src/
│   └─ clases/
│       ├── Matriz.java      clase abstracta (estructura base)
│       ├── OperacionesMatriz.java  interface (contrato de las 10 actividades)
│       ├── CMatrix.java     clase principal de lógica
│       └─ AplicaMatriz.java clase ejecutable con el menú
```

4.1. Diagrama de clases



```
+ main(String[]) : void
+ menu() : void
```

Nota de diseño: la interface `OperacionesMatriz` se relaciona con `CMatriz` mediante una implementación (línea punteada con triángulo en notación UML). Obliga a la clase a definir los doce métodos del contrato; si alguno faltara, el compilador de NetBeans marcaría un error, lo que constituye una garantía de que todas las actividades fueron desarrolladas.

4.2. Clase abstracta `Matriz` (fragmento)

```
public abstract class Matriz {

    // ===== Atributos privados (ENCAPSULAMIENTO) =====
    private int[][] matriz;
    private int tamano;

    /** Metodo ABSTRACTO: la subclase esta obligada a implementarlo. */
    public abstract void escritura();

    /** Getter de un elemento especifico de la matriz. */
    public int getElemento(int fila, int columna) {
        return matriz[fila][columna];
    }

    /** Setter de un elemento especifico de la matriz. */
    public void setElemento(int fila, int columna, int valor) {
        matriz[fila][columna] = valor;
    }

    /**
     * UNICO objeto Scanner de todo el programa: usar dos Scanner sobre
     * System.in provoca perdida de datos y excepciones en ejecucion.
     */
    public static final Scanner TECLADO = new Scanner(System.in);

    /** Lee un entero de forma segura: si el usuario escribe letras, vuelve a preguntar. */
    public static int leerEntero(String mensaje) {
        int valor = 0;
        boolean valido = false;
        while (!valido) {
            System.out.print(mensaje);
            try {
                valor = Integer.parseInt(TECLADO.nextLine().trim());
                valido = true;
            } catch (NumberFormatException e) {
                System.out.println(">> Dato no valido. Debe ingresar un numero entero.");
            }
        }
        return valor;
    }

    /** Reserva memoria para una matriz cuadrada de n x n. */
    public final void creaArreglo(int valor) {
        if (valor < 0) { valor = 0; }
        this.tamano = valor;
        this.matriz = new int[valor][valor];
    }

    /** Valida que la matriz exista antes de operar sobre ella. */
    public boolean validarMatriz() {
```

```

        if (estaVacia()) {
            System.out.println(">> ERROR: primero debe crear la matriz (opcion 1).");
            return false;
        }
        return true;
    }
}

```

4.3. Interface OperacionesMatriz (fragmento)

```

public interface OperacionesMatriz {

    int eliminarElemento(int x); // Actividad 1
    int modificarElemento(int x, int nuevoValor); // Actividad 2
    int sumarElementos(); // Actividad 3
    int obtenerMenor(); // Actividad 4
    int obtenerMayor(); // Actividad 4
    double obtenerPromedio(); // Actividad 4
    int sumaExtremosL(); // Actividad 5
    int sumaExtremosJ(); // Actividad 6
    int sumaExtremosU(); // Actividad 7
    void visualizarMenorPorFila(); // Actividad 8
    int contarPares(); // Actividad 9
    void rotarFilasDerecha(); // Actividad 10
}

```

5. Metodología y matriz de prueba

Para que todas las evidencias sean comparables entre sí, las diez actividades fueron probadas sobre una **misma matriz de 4×4** . Con el fin de agilizar las pruebas se incorporó la opción 18 del menú, *Cargar matriz de ejemplo*, que llena la matriz automáticamente y evita digitar los dieciséis valores en cada ejecución. Los datos fueron elegidos deliberadamente: mezclan pares e impares, incluyen un valor mínimo y un valor máximo bien diferenciados y no contienen repeticiones, lo que facilita la verificación manual.

	c1	c2	c3	c4
f1	8	3	12	5
f2	7	20	1	9
f3	4	15	6	11
f4	2	10	14	13

Matriz de prueba utilizada en todas las evidencias

Característica	Valor	Observación
Dimensión	4×4	16 elementos en total
Suma de los elementos	140	Verificada en la actividad 3
Valor menor / mayor	1 / 20	Ubicados en la fila 2
Cantidad de pares	8	La mitad de los elementos

5.1. Menú de opciones del programa

El menú fue reorganizado en tres bloques para facilitar su uso: configuración, operaciones base del trabajo anterior y las diez actividades desarrolladas en este informe.

Ejecución de AplicaMatriz.java - Menú principal

MENU DE OPCIONES - MATRICES

CONFIGURACION

- 1.- Crear el Arreglo (matriz $n \times n$)
- 2.- Ingresar Datos
- 3.- Mostrar Datos

OPERACIONES BASE

- 4.- Sumar Filas
- 5.- Sumar Columnas
- 6.- Sumar Diagonal Mayor
- 7.- Visualizar el contenido de una posicion

ACTIVIDADES DESARROLLADAS

- 8.- Eliminar un elemento X
- 9.- Modificar un elemento X
- 10.- Sumar todos los elementos
- 11.- Menor, Mayor y Promedio
- 12.- Suma de extremos en forma de L
- 13.- Suma de extremos en forma de J
- 14.- Suma de extremos en forma de U
- 15.- Menor numero de cada fila
- 16.- Contar numeros pares

```
17.- Mover filas una posicion a la derecha
-----
18.- Cargar matriz de ejemplo (4x4)
0.- Salir
=====
```

Figura 1. Menú principal del programa en ejecución.

5.2. Carga de la matriz de prueba (opción 18)

Salida - Opción 18: cargar matriz de ejemplo

Matriz de ejemplo 4x4 cargada correctamente.

Listado

=====

8	3	12	5
7	20	1	9
4	15	6	11
2	10	14	13

Figura 2. Matriz de prueba cargada y visualizada con el método `escritura()`.

1 Eliminar un elemento "x" de la matriz

ENUNCIADO DE LA ACTIVIDAD

Elaborar un método para eliminar un elemento "x", de la matriz creada anteriormente.

Análisis del problema

En Java una matriz declarada como `int[n][n]` es una estructura **estática**: el espacio en memoria se reserva una sola vez y no es posible quitar físicamente una celda sin reconstruir todo el arreglo. Por ello la eliminación se implementa como una **eliminación lógica**: se recorre la matriz, se compara cada elemento con el valor buscado y, cuando coincide, se le asigna el valor `0`, que es el elemento neutro de la suma y por lo tanto no altera los cálculos posteriores.

El método recorre la matriz con un **doble bucle for** (uno para las filas y otro para las columnas), informa la posición exacta de cada coincidencia y devuelve la cantidad total de elementos eliminados, de modo que la clase principal pueda mostrar un mensaje adecuado.

	c1	c2	c3	c4
f1	8	3	12	5
f2	7	20	1	9
f3	4	15	6	11
f4	2	10	14	13



	c1	c2	c3	c4
f1	8	3	0	5
f2	7	20	1	9
f3	4	15	6	11
f4	2	10	14	13

Matriz original (se busca el 12)

Resultado: la celda queda en 0

Representación gráfica del proceso realizado sobre la matriz de prueba.

Código desarrollado

```
/**
 * Elimina logicamente todas las apariciones del valor x.
 * En Java una matriz tiene tamaño FIJO, por lo tanto no se puede quitar
 * una celda; la eliminación se realiza asignando el valor neutro 0.
 *
 * @param x valor a eliminar
 * @return cantidad de elementos eliminados
 */
@Override
public int eliminarElemento(int x) {
    int contador = 0;
    for (int i = 0; i < getTamano(); i++) {
        for (int j = 0; j < getTamano(); j++) {
            if (getElemento(i, j) == x) {
                setElemento(i, j, 0);
                contador++;
                System.out.println("Elemento " + x + " eliminado de la fila "
                    + (i + 1) + ", columna " + (j + 1));
            }
        }
    }
    if (contador == 0) {
        System.out.println("El elemento " + x + " no existe en la matriz.");
    } else {
        System.out.println("Total de elementos eliminados: " + contador);
    }
}
```

```
return contador;  
}
```

Explicación paso a paso

Paso	Acción	Descripción
1	Declarar el contador	Se inicializa <code>contador = 0</code> para llevar la cuenta de cuántas coincidencias se encuentran.
2	Recorrer las filas	El bucle externo <code>for (int i = 0; i < getTamano(); i++)</code> avanza fila por fila.
3	Recorrer las columnas	El bucle interno <code>for (int j = 0; j < getTamano(); j++)</code> visita cada columna de la fila actual.
4	Comparar el valor	La condición <code>if (getElemento(i, j) == x)</code> pregunta si la celda contiene el valor buscado.
5	Eliminar y registrar	Si coincide: <code>setElemento(i, j, 0)</code> borra el dato, se incrementa el contador y se informa la posición en base 1 (<code>i+1, j+1</code>).
6	Informar el resultado	Al terminar los bucles, si <code>contador == 0</code> se avisa que el elemento no existe; caso contrario se muestra el total eliminado.
7	Devolver el contador	<code>return contador;</code> permite que otras clases sepan cuántos elementos fueron eliminados.

Evidencia de ejecución

Datos ingresados: **X = 12**

Salida - Actividad 1

```
Ingrese el elemento X a eliminar: 12  
Elemento 12 eliminado de la fila 1, columna 3  
Total de elementos eliminados: 1
```

Matriz resultante:

Listado

=====

```
8      3      0      5  
7      20     1      9  
4      15     6     11  
2      10     14     13
```

Figura 3. Resultado obtenido en la consola de NetBeans.

Verificación del resultado

Resultado correcto.

El valor **12** se encontraba únicamente en la **fila 1, columna 3**. Después de la ejecución esa celda muestra **0** y el resto de la matriz permanece intacto, tal como se esperaba. La suma total pasa de **140** a **128**, es decir, exactamente 12 unidades menos.

2 Modificar un elemento "x" de la matriz

ENUNCIADO DE LA ACTIVIDAD

Elaborar un método para modificar un elemento "x" de la matriz creada anteriormente.

Análisis del problema

Este método es una variante del anterior: en lugar de asignar `0`, escribe un **nuevo valor** proporcionado por el usuario. Recibe dos parámetros —el valor que se desea cambiar (`x`) y el valor que ocupará su lugar (`nuevoValor`)— lo que demuestra el **paso de parámetros** y la reutilización de la misma estructura de recorrido.

Se modifican **todas las apariciones** del valor. Si se quisiera cambiar solo la primera coincidencia, bastaría con agregar una bandera booleana o una instrucción `break` tras el primer reemplazo.

	c1	c2	c3	c4
f1	8	3	12	5
f2	7	20	1	9
f3	4	15	6	11
f4	2	10	14	13



	c1	c2	c3	c4
f1	8	3	12	5
f2	7	99	1	9
f3	4	15	6	11
f4	2	10	14	13

Antes: el elemento 20

Después: reemplazado por 99

Representación gráfica del proceso realizado sobre la matriz de prueba.

Código desarrollado

```
/**
 * Reemplaza todas las apariciones del valor x por un nuevo valor.
 */
@param x          valor actual que se desea cambiar
@param nuevoValor valor que se colocara en su lugar
@return cantidad de elementos modificados
*/
@Override
public int modificarElemento(int x, int nuevoValor) {
    int contador = 0;
    for (int i = 0; i < getTamano(); i++) {
        for (int j = 0; j < getTamano(); j++) {
            if (getElemento(i, j) == x) {
                setElemento(i, j, nuevoValor);
                contador++;
                System.out.println("Fila " + (i + 1) + ", columna " + (j + 1)
                    + ": " + x + " -> " + nuevoValor);
            }
        }
    }
    if (contador == 0) {
        System.out.println("El elemento " + x + " no existe en la matriz.");
    } else {
        System.out.println("Total de elementos modificados: " + contador);
    }
    return contador;
}
```

Explicación paso a paso

Paso	Acción	Descripción
1	Recibir los parámetros	La firma <code>modificarElemento(int x, int nuevoValor)</code> recibe el dato viejo y el dato nuevo.
2	Inicializar el contador	<code>contador = 0</code> registrará cuántos reemplazos se realizan.
3	Recorrido completo	Los dos bucles <code>for</code> anidados visitan las $n \times n$ posiciones de la matriz.
4	Detectar la coincidencia	<code>if (getElemento(i, j) == x)</code> identifica la celda que se debe cambiar.
5	Reemplazar el valor	<code>setElemento(i, j, nuevoValor)</code> escribe el nuevo dato usando el <code>setter</code> , respetando el encapsulamiento.
6	Mostrar el cambio	Se imprime la posición junto con el cambio realizado en el formato <code>viejo -> nuevo</code> .
7	Validar y retornar	Si no hubo coincidencias se informa al usuario; finalmente se devuelve el total de modificaciones.

Evidencia de ejecución

Datos ingresados: **X = 20** | **Nuevo valor = 99**

```
Salida - Actividad 2

Ingrese el elemento X a modificar: 20
Ingrese el nuevo valor: 99
Fila 2, columna 2: 20 -> 99
Total de elementos modificados: 1

Matriz resultante:
Listado
=====
8      3      12      5
7      99      1      9
4      15      6      11
2      10      14      13
```

Figura 4. Resultado obtenido en la consola de NetBeans.

Verificación del resultado

Resultado correcto.

El valor **20** ocupaba la **fila 2, columna 2**. Tras la ejecución esa posición contiene **99** y ninguna otra celda se vio afectada, confirmando que el recorrido y la condición de comparación funcionan correctamente.

3 Sumar los elementos de la matriz

ENUNCIADO DE LA ACTIVIDAD

Elaborar un método para sumar los elementos de la matriz.

Análisis del problema

La suma total se obtiene con la técnica del **acumulador**: una variable inicializada en `0` que va incrementándose con el contenido de cada celda visitada. Es fundamental declarar el acumulador **antes** de los bucles; si se declarara dentro del bucle interno se reiniciaría en cada vuelta y el resultado sería incorrecto.

A diferencia de los métodos anteriores, `sumarElementos()` **devuelve un valor** (`int`) en lugar de imprimirlo. Esto permite reutilizarlo en otros métodos —por ejemplo, en el cálculo del promedio— y separa la lógica del negocio de la presentación en pantalla, que queda a cargo del método `visualizarSumaTotal()`.

	c1	c2	c3	c4
f1	8	3	12	5
f2	7	20	1	9
f3	4	15	6	11
f4	2	10	14	13

Se recorren y acumulan las 16 celdas

Representación gráfica del proceso realizado sobre la matriz de prueba.

Código desarrollado

```
@Override
public int sumarElementos() {
    int suma = 0;
    for (int i = 0; i < getTamano(); i++) {
        for (int j = 0; j < getTamano(); j++) {
            suma += getElemento(i, j);
        }
    }
    return suma;
}

/** Metodo de presentacion: muestra la suma total en pantalla. */
public void visualizarSumaTotal() {
    System.out.println("La suma de todos los elementos de la matriz es: " + sumarElementos());
}
```

Explicación paso a paso

Paso	Acción	Descripción
1	Crear el acumulador	<code>int suma = 0;</code> se declara fuera de los bucles para conservar su valor.
2	Recorrer filas y columnas	Los bucles anidados garantizan que se visiten las 16 posiciones de una matriz 4×4 .
3	Acumular	<code>suma += getElemento(i, j);</code> equivale a <code>suma = suma + elemento.</code>

4	Devolver el total	<code>return suma;</code> entrega el resultado a quien haya invocado el método.
5	Mostrar en pantalla	El método <code>visualizarSumaTotal()</code> invoca al anterior y presenta el mensaje al usuario.

Evidencia de ejecución

Datos ingresados: —

```
Salida - Actividad 3

La suma de todos los elementos de la matriz es: 140
```

Figura 5. Resultado obtenido en la consola de NetBeans.

Verificación del resultado

Resultado correcto.

Comprobación manual sumando fila por fila:

Fila	Elementos	Suma parcial
Fila 1	8 + 3 + 12 + 5	28
Fila 2	7 + 20 + 1 + 9	37
Fila 3	4 + 15 + 6 + 11	36
Fila 4	2 + 10 + 14 + 13	39
TOTAL		140

28 + 37 + 36 + 39 = **140**, valor idéntico al mostrado por el programa.

4 Número menor, número mayor y promedio

ENUNCIADO DE LA ACTIVIDAD

Elaborar un método para hallar el número menor, el número mayor y el promedio de los elementos de la matriz.

Análisis del problema

Para buscar extremos se aplica la técnica de **comparación sucesiva**. El punto clave es **inicializar la variable con el primer elemento real de la matriz** (`getElemento(0,0)`) y no con `0`: si la matriz contuviera únicamente números positivos, inicializar el menor en 0 devolvería siempre 0, que es un error muy común.

El promedio se obtiene dividiendo la suma total entre la cantidad de elementos (`n × n`). Como ambos operandos son enteros, Java realizaría una **división entera** y descartaría los decimales; por eso se aplica una **conversión explícita de tipo (casting)** con `(double)`. Además, se reutiliza `sumarElementos()` de la actividad anterior, evidenciando la **reutilización de código**.

	c1	c2	c3	c4
f1	8	3	12	5
f2	7	20	1	9
f3	4	15	6	11
f4	2	10	14	13

Menor = 1 (fila 2, col 3)

	c1	c2	c3	c4
f1	8	3	12	5
f2	7	20	1	9
f3	4	15	6	11
f4	2	10	14	13

Mayor = 20 (fila 2, col 2)

Representación gráfica del proceso realizado sobre la matriz de prueba.

Código desarrollado

```
@Override
public int obtenerMenor() {
    int menor = getElemento(0, 0);
    for (int i = 0; i < getTamano(); i++) {
        for (int j = 0; j < getTamano(); j++) {
            if (getElemento(i, j) < menor) {
                menor = getElemento(i, j);
            }
        }
    }
    return menor;
}

@Override
public int obtenerMayor() {
    int mayor = getElemento(0, 0);
    for (int i = 0; i < getTamano(); i++) {
        for (int j = 0; j < getTamano(); j++) {
            if (getElemento(i, j) > mayor) {
                mayor = getElemento(i, j);
            }
        }
    }
    return mayor;
}
```

```

}

@Override
public double obtenerPromedio() {
    int cantidad = getTamano() * getTamano();
    return (double) sumarElementos() / cantidad;
}

@Override
public void visualizarEstadisticas() {
    System.out.println("El numero MENOR de la matriz es : " + obtenerMenor());
    System.out.println("El numero MAYOR de la matriz es : " + obtenerMayor());
    System.out.printf("El PROMEDIO de la matriz es      : %.2f%n", obtenerPromedio());
}

```

Explicación paso a paso

Paso	Acción	Descripción
1	Inicializar con el primer elemento	<code>int menor = getElemento(0, 0);</code> establece un punto de partida válido.
2	Recorrer toda la matriz	Los bucles anidados comparan el elemento actual con el mejor candidato encontrado.
3	Actualizar el menor	Si <code>getElemento(i, j) < menor</code> , el candidato se convierte en el nuevo menor.
4	Actualizar el mayor	De forma simétrica, <code>if (getElemento(i, j) > mayor)</code> guarda el valor más alto.
5	Calcular la cantidad	<code>int cantidad = getTamano() * getTamano();</code> obtiene el número total de celdas.
6	Aplicar el casting	<code>(double) sumarElementos() / cantidad</code> fuerza la división real con decimales.
7	Presentar con formato	<code>System.out.printf("%.2f%n", ...)</code> muestra el promedio con dos decimales.

Evidencia de ejecución

Datos ingresados: —

Salida - Actividad 4

```

El numero MENOR de la matriz es : 1
El numero MAYOR de la matriz es : 20
El PROMEDIO de la matriz es      : 8.75

```

Figura 6. Resultado obtenido en la consola de NetBeans.

Verificación del resultado

Resultado correcto.

Recorriendo visualmente la matriz, el valor más pequeño es **1** y el más grande es **20**. El promedio se calcula como:

$$\text{Promedio} = 140 \div 16 = 8,75$$

El programa muestra **8.75**, con lo cual los tres resultados quedan verificados.

5 Suma de los contenidos extremos que forman la letra L

ENUNCIADO DE LA ACTIVIDAD

Elaborar un método para visualizar la suma de los contenidos extremos que forman la letra L, en la matriz.

Análisis del problema

La letra **L** está formada por dos trazos: un **palo vertical** que corresponde a la **primera columna** completa, y una **base horizontal** que corresponde a la **última fila**. Ambos trazos se cruzan en la esquina inferior izquierda —la posición `[n-1][0]`—, por lo que ese elemento se sumaría **dos veces** si no se controla.

La solución consiste en recorrer la primera columna completa (`i` de 0 a `n-1`) y luego la última fila **empezando en la columna 1** (`j` de 1 a `n-1`), evitando así repetir la esquina. Se emplean dos bucles `for` simples e independientes, no anidados, porque solo se visitan los bordes y no toda la matriz.

	c1	c2	c3	c4
f1	8	3	12	5
f2	7	20	1	9
f3	4	15	6	11
f4	2	10	14	13

Celdas que forman la L

Representación gráfica del proceso realizado sobre la matriz de prueba.

Código desarrollado

```
/**
 * Forma de la L (ejemplo 4x4):
 *   X . . .
 *   X . . .
 *   X . . .
 *   X X X X
 * Primera columna completa + ultima fila (sin repetir la esquina).
 */
@Override
public int sumaExtremosL() {
    int n = getTamano();
    int suma = 0;
    for (int i = 0; i < n; i++) {           // palo vertical: primera columna
        suma += getElemento(i, 0);
    }
    for (int j = 1; j < n; j++) {           // base horizontal: ultima fila
        suma += getElemento(n - 1, j);
    }
    return suma;
}

public void visualizarSumaL() {
    System.out.println("La suma de los elementos que forman la letra L es: " + sumaExtremosL());
}
```

Explicación paso a paso

Paso	Acción	Descripción
1	Obtener el tamaño	<code>int n = getTamano();</code> se guarda en una variable local para simplificar la escritura.
2	Inicializar el acumulador	<code>int suma = 0;</code> almacenará el total de los extremos.
3	Recorrer el palo vertical	<code>for (int i = 0; i < n; i++) suma += getElemento(i, 0);</code> suma la primera columna completa.
4	Recorrer la base	<code>for (int j = 1; j < n; j++) suma += getElemento(n-1, j);</code> suma la última fila desde la columna 1.
5	Evitar la duplicación	Iniciar el segundo bucle en <code>j = 1</code> impide que la esquina <code>[n-1][0]</code> se sume dos veces.
6	Devolver y visualizar	El método retorna la suma y <code>visualizarSumaL()</code> la muestra en pantalla.

Evidencia de ejecución

Datos ingresados: —

Salida - Actividad 5

La suma de los elementos que forman la letra L es: 58

Figura 7. Resultado obtenido en la consola de NetBeans.

Verificación del resultado

Resultado correcto.

Verificación manual de los elementos resaltados:

- Palo vertical (columna 1): $8 + 7 + 4 + 2 = 21$
- Base (fila 4, desde la columna 2): $10 + 14 + 13 = 37$

$$21 + 37 = 58$$

Coincide con el resultado obtenido en la ejecución del programa.

6 Suma de los contenidos extremos que forman la letra J

ENUNCIADO DE LA ACTIVIDAD

Elaborar un método para visualizar la suma de los contenidos extremos que forman la letra J, en la matriz.

Análisis del problema

La letra J es la imagen especular de la L: el palo vertical se ubica en la **última columna** y la base sigue siendo la **última fila**. El punto de cruce es ahora la esquina inferior derecha `[n-1][n-1]`.

Para no duplicar esa esquina, el primer bucle recorre la última columna completa y el segundo recorre la última fila **hasta la penúltima columna** (`j < n-1`). Comparar este método con el de la letra L permite apreciar cómo un pequeño cambio en los índices modifica por completo la figura recorrida.

	c1	c2	c3	c4
f1	8	3	12	5
f2	7	20	1	9
f3	4	15	6	11
f4	2	10	14	13

Celdas que forman la J

Representación gráfica del proceso realizado sobre la matriz de prueba.

Código desarrollado

```
/**
 * Forma de la J (ejemplo 4x4):
 *   . . . X
 *   . . . X
 *   . . . X
 *   X X X X
 * Última columna completa + última fila (sin repetir la esquina).
 */
@Override
public int sumaExtremosJ() {
    int n = getTamano();
    int suma = 0;
    for (int i = 0; i < n; i++) {           // palo vertical: ultima columna
        suma += getElemento(i, n - 1);
    }
    for (int j = 0; j < n - 1; j++) {       // base horizontal: ultima fila
        suma += getElemento(n - 1, j);
    }
    return suma;
}

public void visualizarSumaJ() {
    System.out.println("La suma de los elementos que forman la letra J es: " + sumaExtremosJ());
}
```

Explicación paso a paso

Paso	Acción	Descripción
1	Obtener el tamaño	<code>int n = getTamano();</code> define los límites de los recorridos.
2	Recorrer el palo vertical derecho	<code>for (int i = 0; i < n; i++) suma += getElemento(i, n-1);</code> suma la última columna.
3	Recorrer la base	<code>for (int j = 0; j < n-1; j++) suma += getElemento(n-1, j);</code> suma la última fila.
4	Excluir la esquina	La condición <code>j < n-1</code> detiene el bucle antes de la esquina ya contabilizada.
5	Retornar el resultado	La suma se devuelve como entero y el método <code>visualizarSumaJ()</code> la imprime.

Evidencia de ejecución

Datos ingresados: —

```
Salida - Actividad 6

La suma de los elementos que forman la letra J es: 64
```

Figura 8. Resultado obtenido en la consola de NetBeans.

Verificación del resultado

Resultado correcto.

Verificación manual:

- Palo vertical (columna 4): $5 + 9 + 11 + 13 = 38$
- Base (fila 4, columnas 1 a 3): $2 + 10 + 14 = 26$

$$38 + 26 = 64$$

El valor calculado a mano coincide con la salida del programa.

7 Suma de los contenidos extremos que forman la letra U

ENUNCIADO DE LA ACTIVIDAD

Elaborar un método para visualizar la suma de los contenidos extremos que forman la letra U, en la matriz.

Análisis del problema

La letra **U** combina **tres trazos**: el palo izquierdo (primera columna), el palo derecho (última columna) y la base (última fila). En este caso existen **dos esquinas** compartidas: la inferior izquierda y la inferior derecha.

La estrategia empleada consiste en recorrer ambos palos **sin incluir la última fila** ($i < n-1$) y luego sumar la **última fila completa**. De esta manera cada celda de la figura se suma exactamente una vez. El método utiliza tres bucles `for` simples, lo que lo mantiene legible y eficiente: solo visita los bordes, no las 16 celdas.

	c1	c2	c3	c4
f1	8	3	12	5
f2	7	20	1	9
f3	4	15	6	11
f4	2	10	14	13

Celdas que forman la U

Representación gráfica del proceso realizado sobre la matriz de prueba.

Código desarrollado

```
/**
 * Forma de la U (ejemplo 4x4):
 *   X . . X
 *   X . . X
 *   X . . X
 *   X X X X
 * Primera columna + ultima columna (sin la ultima fila) + ultima fila completa.
 */
@Override
public int sumaExtremosU() {
    int n = getTamano();
    int suma = 0;
    for (int i = 0; i < n - 1; i++) { // palo izquierdo
        suma += getElemento(i, 0);
    }
    for (int i = 0; i < n - 1; i++) { // palo derecho
        suma += getElemento(i, n - 1);
    }
    for (int j = 0; j < n; j++) { // base de la U
        suma += getElemento(n - 1, j);
    }
    return suma;
}

public void visualizarSumaU() {
    System.out.println("La suma de los elementos que forman la letra U es: " + sumaExtremosU());
}
```

Explicación paso a paso

Paso	Acción	Descripción
1	Obtener el tamaño	Se guarda n para controlar los tres recorridos.
2	Palo izquierdo	<code>for (int i = 0; i < n-1; i++) suma += getElemento(i, 0);</code> recorre la primera columna sin llegar a la última fila.
3	Palo derecho	<code>for (int i = 0; i < n-1; i++) suma += getElemento(i, n-1);</code> hace lo mismo con la última columna.
4	Base de la U	<code>for (int j = 0; j < n; j++) suma += getElemento(n-1, j);</code> suma la última fila completa, incluidas ambas esquinas.
5	Verificar la no duplicación	Al excluir la última fila de los dos primeros bucles, ninguna celda se suma dos veces.
6	Devolver y mostrar	Se retorna el acumulado y <code>visualizarSumaU()</code> lo presenta en consola.

Evidencia de ejecución

Datos ingresados: —

Salida - Actividad 7

La suma de los elementos que forman la letra U es: 83

Figura 9. Resultado obtenido en la consola de NetBeans.

Verificación del resultado

Resultado correcto.

Verificación manual:

- Palo izquierdo (filas 1 a 3 de la columna 1): $8 + 7 + 4 = 19$
- Palo derecho (filas 1 a 3 de la columna 4): $5 + 9 + 11 = 25$
- Base (fila 4 completa): $2 + 10 + 14 + 13 = 39$

$$19 + 25 + 39 = 83$$

Resultado idéntico al de la ejecución.

8 Menor número de cada fila

ENUNCIADO DE LA ACTIVIDAD

Elaborar un método para visualizar el menor número de cada fila, en la matriz.

Análisis del problema

A diferencia de la actividad 4, donde se buscaba un único mínimo global, aquí se requiere un **mínimo por cada fila**. La clave está en **reiniciar la variable `menorFila` dentro del bucle externo**: cada vez que se inicia una fila nueva, el candidato se restablece con el primer elemento de esa fila.

El bucle interno comienza en `j = 1` porque la posición 0 ya fue tomada como valor inicial, lo que evita una comparación innecesaria. Adicionalmente se guarda la **columna** donde se encontró el mínimo, entregando una salida más informativa al usuario.

	c1	c2	c3	c4
f1	8	3	12	5
f2	7	20	1	9
f3	4	15	6	11
f4	2	10	14	13

Mínimo de cada fila resaltado

Representación gráfica del proceso realizado sobre la matriz de prueba.

Código desarrollado

```
@Override
public void visualizarMenorPorFila() {
    for (int i = 0; i < getTamano(); i++) {
        int menorFila = getElemento(i, 0);
        int columna = 0;
        for (int j = 1; j < getTamano(); j++) {
            if (getElemento(i, j) < menorFila) {
                menorFila = getElemento(i, j);
                columna = j;
            }
        }
        System.out.println("El menor numero de la fila " + (i + 1) + " es: "
            + menorFila + " (columna " + (columna + 1) + ")");
    }
}
```

Explicación paso a paso

Paso	Acción	Descripción
1	Recorrer las filas	El bucle externo <code>for (int i = 0; ...)</code> procesa una fila por vuelta.
2	Reiniciar el candidato	<code>int menorFila = getElemento(i, 0);</code> se ejecuta al inicio de cada fila; este es el paso decisivo.
3	Guardar la posición	La variable <code>columna</code> almacena el índice donde se encontró el mínimo.

4	Comparar desde la columna 1	El bucle interno inicia en $j = 1$ porque la columna 0 ya está considerada.
5	Actualizar el mínimo	Si el elemento actual es menor, se actualizan <code>menorFila</code> y <code>columna</code> .
6	Mostrar el resultado	Al terminar cada fila se imprime su menor valor junto con su ubicación.

Evidencia de ejecución

Datos ingresados: —

```
Salida - Actividad 8

El menor numero de la fila 1 es: 3 (columna 2)
El menor numero de la fila 2 es: 1 (columna 3)
El menor numero de la fila 3 es: 4 (columna 1)
El menor numero de la fila 4 es: 2 (columna 1)
```

Figura 10. Resultado obtenido en la consola de NetBeans.

Verificación del resultado

Resultado correcto.

Fila	Elementos	Menor	Columna
Fila 1	8, 3, 12, 5	3	2
Fila 2	7, 20, 1, 9	1	3
Fila 3	4, 15, 6, 11	4	1
Fila 4	2, 10, 14, 13	2	1

Los cuatro valores coinciden con los mostrados por el programa.

9 Contar los números pares de la matriz

ENUNCIADO DE LA ACTIVIDAD

Elaborar un método para contar los números pares en la matriz.

Análisis del problema

Un número es **par** cuando al dividirlo entre 2 el residuo es cero. En Java el residuo se obtiene con el **operador módulo** `%`, por lo que la condición es `if (getElemento(i, j) % 2 == 0)`.

Se aplica la técnica del **contador**: una variable que se incrementa en 1 cada vez que se cumple la condición. Como el total de celdas es conocido ($n \times n$), la cantidad de impares se deduce por diferencia, sin necesidad de un segundo recorrido. Conviene señalar que el operador módulo también funciona correctamente con números negativos: `-4 % 2` es 0 (par) y `-3 % 2` es -1 (distinto de 0, impar).

	c1	c2	c3	c4
f1	8	3	12	5
f2	7	20	1	9
f3	4	15	6	11
f4	2	10	14	13

Números pares resaltados (8 en total)

Representación gráfica del proceso realizado sobre la matriz de prueba.

Código desarrollado

```
@Override
public int contarPares() {
    int contador = 0;
    for (int i = 0; i < getTamano(); i++) {
        for (int j = 0; j < getTamano(); j++) {
            if (getElemento(i, j) % 2 == 0) {
                contador++;
            }
        }
    }
    return contador;
}

public void visualizarPares() {
    System.out.println("La cantidad de numeros PARES en la matriz es : " + contarPares());
    System.out.println("La cantidad de numeros IMPARES en la matriz es: "
        + (getTamano() * getTamano() - contarPares()));
}
```

Explicación paso a paso

Paso	Acción	Descripción
1	Inicializar el contador	<code>int contador = 0;</code> se declara fuera de los bucles.
2	Recorrer la matriz	Los bucles anidados visitan las $n \times n$ posiciones.

3	Evaluar la paridad	<code>getElemento(i,j) % 2 == 0</code> devuelve <code>true</code> cuando el número es par.
4	Incrementar	Si la condición se cumple se ejecuta <code>contador++</code> .
5	Retornar el total	El método devuelve la cantidad de pares encontrados.
6	Calcular los impares	En <code>visualizarPares()</code> se obtienen los impares como <code>n*n - contarPares()</code> .

Evidencia de ejecución

Datos ingresados: —

```
Salida - Actividad 9

La cantidad de numeros PARES en la matriz es : 8
La cantidad de numeros IMPARES en la matriz es: 8
```

Figura 11. Resultado obtenido en la consola de NetBeans.

Verificación del resultado

Resultado correcto.

Los números pares presentes en la matriz son: **8, 12, 20, 4, 6, 2, 10 y 14**, es decir, **8 elementos**. Como la matriz tiene 16 celdas, los impares son $16 - 8 = 8$ (3, 5, 7, 1, 9, 15, 11 y 13). Ambos valores coinciden con la salida del programa.

10 Mover las filas una posición a la derecha

ENUNCIADO DE LA ACTIVIDAD

Elaborar un método para mover los contenidos de las filas de la matriz una posición a la derecha; el último elemento de cada fila pasará a la primera posición de la misma.

Análisis del problema

Se trata de una **rotación circular** aplicada fila por fila. El problema principal es que, si se copiara cada elemento hacia la derecha empezando por la izquierda, se **sobrescribirían** los datos aún no procesados y toda la fila terminaría con el mismo valor repetido.

La solución aplica dos precauciones: primero se **guarda el último elemento** en una variable auxiliar, y luego el desplazamiento se realiza **de derecha a izquierda** (`for (int j = n-1; j > 0; j--)`), copiando cada elemento desde la posición anterior. Finalmente, el valor auxiliar se coloca en la primera posición, cerrando el ciclo.

	c1	c2	c3	c4
f1	8	3	12	5
f2	7	20	1	9
f3	4	15	6	11
f4	2	10	14	13



	c1	c2	c3	c4
f1	5	8	3	12
f2	9	7	20	1
f3	11	4	15	6
f4	13	2	10	14

Matriz original

Filas desplazadas a la derecha

Representación gráfica del proceso realizado sobre la matriz de prueba.

Código desarrollado

```
/**
 * Desplaza cada fila una posición a la derecha (rotación circular):
 * el último elemento de la fila pasa a ser el primero.
 */
@Override
public void rotarFilasDerecha() {
    int n = getTamano();
    for (int i = 0; i < n; i++) {
        int ultimo = getElemento(i, n - 1); // 1) se guarda el ultimo
        for (int j = n - 1; j > 0; j--) { // 2) se desplaza de derecha a izquierda
            setElemento(i, j, getElemento(i, j - 1));
        }
        setElemento(i, 0, ultimo); // 3) el ultimo pasa al inicio
    }
    System.out.println("Filas desplazadas una posición a la derecha.");
}
```

Explicación paso a paso

Paso	Acción	Descripción
1	Recorrer fila por fila	El bucle externo <code>for (int i = 0; i < n; i++)</code> procesa una fila completa en cada vuelta.

2	Guardar el último elemento	<code>int ultimo = getElemento(i, n-1);</code> conserva el dato que se perdería al desplazar.
3	Desplazar en sentido inverso	<code>for (int j = n-1; j > 0; j--)</code> avanza desde la última hacia la segunda columna.
4	Copiar el vecino izquierdo	<code>setElemento(i, j, getElemento(i, j-1));</code> mueve cada valor una posición a la derecha.
5	Cerrar el ciclo	<code>setElemento(i, 0, ultimo);</code> coloca el elemento guardado en la primera posición.
6	Repetir en todas las filas	El proceso se repite hasta completar las n filas de la matriz.

Evidencia de ejecución

Datos ingresados: —

```
Salida - Actividad 10

Matriz ORIGINAL:
Listado
=====
8      3      12      5
7      20     1      9
4      15     6      11
2      10     14     13
Filas desplazadas una posicion a la derecha.

Matriz DESPLAZADA:
Listado
=====
5      8      3      12
9      7      20     1
11     4      15     6
13     2      10     14
```

Figura 12. Resultado obtenido en la consola de NetBeans.

Verificación del resultado

Resultado correcto.

Fila	Antes	Después
Fila 1	8, 3, 12, 5	5, 8, 3, 12
Fila 2	7, 20, 1, 9	9, 7, 20, 1
Fila 3	4, 15, 6, 11	11, 4, 15, 6
Fila 4	2, 10, 14, 13	13, 2, 10, 14

En las cuatro filas el último elemento pasó correctamente a la primera posición y los demás se desplazaron una casilla a la derecha, sin pérdida ni duplicación de datos.

16. Resumen general de resultados

La siguiente tabla consolida las diez actividades desarrolladas, el método que las resuelve y el resultado obtenido sobre la matriz de prueba. Todos los valores fueron contrastados con el cálculo manual presentado en cada sección.

Nº	Método implementado	Descripción	Resultado
1	<code>eliminarElemento(12)</code>	Eliminación lógica del valor indicado	1 eliminado
2	<code>modificarElemento(20, 99)</code>	Reemplazo del valor por uno nuevo	1 modificado
3	<code>sumarElementos()</code>	Suma de los 16 elementos	140
4	<code>visualizarEstadisticas()</code>	Menor, mayor y promedio	1 / 20 / 8,75
5	<code>sumaExtremosL()</code>	Primera columna + última fila	58
6	<code>sumaExtremosJ()</code>	Última columna + última fila	64
7	<code>sumaExtremosU()</code>	Ambas columnas + última fila	83
8	<code>visualizarMenorPorFila()</code>	Mínimo de cada fila	3, 1, 4, 2
9	<code>contarPares()</code>	Cantidad de números pares	8
10	<code>rotarFilasDerecha()</code>	Desplazamiento circular de filas	Correcto

Comparación de las figuras L, J y U

Las actividades 5, 6 y 7 comparten la misma idea —recorrer los bordes de la matriz— pero difieren en los índices utilizados. La comparación lado a lado ayuda a comprender el manejo de índices:

8	3	12	5
7	20	1	9
4	15	6	11
2	10	14	13

Letra L = 58

8	3	12	5
7	20	1	9
4	15	6	11
2	10	14	13

Letra J = 64

8	3	12	5
7	20	1	9
4	15	6	11
2	10	14	13

Letra U = 83

Figura	Trazos que la componen	Control de esquinas
L	Columna 1 completa + última fila	El segundo bucle inicia en $j = 1$
J	Última columna completa + última fila	El segundo bucle termina en $j < n - 1$
U	Columna 1 + última columna + última fila	Los dos primeros bucles terminan en $i < n - 1$

Observación importante: en las tres figuras el riesgo principal es sumar dos veces las celdas donde se cruzan los trazos. La solución siempre consiste en ajustar los límites de uno de los bucles, nunca en restar el valor después, ya que esto último complicaría innecesariamente el código y sería más propenso a errores.

Anexo B: código fuente completo

Se presenta el contenido íntegro de los cuatro archivos que conforman el proyecto, en el mismo estado en que se ejecutaron las evidencias mostradas en este informe.

Matriz.java — clase abstracta con la estructura de datos

```
package clases;

import java.util.Scanner;

/**
 * Clase ABSTRACTA que representa la estructura base de una matriz cuadrada.
 *
 * Pilares de POO aplicados:
 * - ABSTRACCION : define QUE hace una matriz (escritura) sin decir COMO.
 * - ENCAPSULAMIENTO: los atributos son private y solo se acceden por get/set.
 * - HERENCIA : CMatrix hereda de esta clase (extends Matriz).
 *
 * @author Taipei Sulla Fernando
 */
public abstract class Matriz {

    // ===== Atributos privados (ENCAPSULAMIENTO) =====
    private int[][] matriz;
    private int tamano;

    /**
     * UNICO objeto Scanner de todo el programa.
     * Usar dos Scanner sobre System.in provoca perdida de datos y
     * excepciones en tiempo de ejecucion, por eso se comparte uno solo.
     */
    public static final Scanner TECLADO = new Scanner(System.in);

    /**
     * Lee un numero entero desde el teclado de forma segura.
     * Si el usuario escribe letras o deja la linea vacia, vuelve a preguntar
     * en lugar de lanzar InputMismatchException y cortar el programa.
     *
     * @param mensaje texto que se muestra al usuario
     * @return el numero entero ingresado
     */
    public static int leerEntero(String mensaje) {
        int valor = 0;
        boolean valido = false;
        while (!valido) {
            System.out.print(mensaje);
            try {
                valor = Integer.parseInt(TECLADO.nextLine().trim());
                valido = true;
            } catch (NumberFormatException e) {
                System.out.println(">> Dato no valido. Debe ingresar un numero entero.");
            }
        }
        return valor;
    }

    // ===== Constructores =====

    /** Constructor por defecto: matriz vacia de tamano 0. */
    public Matriz() {
        this.tamano = 0;
    }
}
```

```

        this.matriz = new int[0][0];
    }

    /** Constructor sobrecargado: crea la matriz de n x n. */
    public Matriz(int tamano) {
        creaArreglo(tamano);
    }

    /** Constructor de copia: duplica el contenido de otra matriz ya existente. */
    public Matriz(int[][] matrizOriginal) {
        this.tamano = matrizOriginal.length;
        this.matriz = new int[tamano][tamano];
        for (int i = 0; i < tamano; i++) {
            System.arraycopy(matrizOriginal[i], 0, this.matriz[i], 0, tamano);
        }
    }

    // ===== Metodo ABSTRACTO (lo implementa obligatoriamente la subclase) =====

    /** Muestra el contenido de la matriz en pantalla. */
    public abstract void escritura();

    // ===== Getters y Setters =====

    public int[][] getMatriz() {
        return matriz;
    }

    public void setMatriz(int[][] matriz) {
        this.matriz = matriz;
        this.tamano = matriz.length;
    }

    public int getTamano() {
        return tamano;
    }

    public void setTamano(int tamano) {
        creaArreglo(tamano);
    }

    /** Getter de un elemento especifico de la matriz. */
    public int getElemento(int fila, int columna) {
        return matriz[fila][columna];
    }

    /** Setter de un elemento especifico de la matriz. */
    public void setElemento(int fila, int columna, int valor) {
        matriz[fila][columna] = valor;
    }

    // ===== Metodos de servicio comunes =====

    /** Reserva memoria para una matriz cuadrada de n x n. */
    public final void creaArreglo(int valor) {
        if (valor < 0) {
            valor = 0;
        }
        this.tamano = valor;
        this.matriz = new int[valor][valor];
    }

    /** Devuelve true si la matriz aun no ha sido creada. */
    public boolean estaVacía() {

```

```

        return tamano == 0;
    }

    /** Valida que la matriz exista antes de operar sobre ella. */
    public boolean validarMatriz() {
        if (estaVacia()) {
            System.out.println(">> ERROR: primero debe crear la matriz (opcion 1) e ingresar datos (opcion 2).");
            return false;
        }
        return true;
    }
}

```

OperacionesMatriz.java — interface con el contrato de las actividades

```

package clases;

/**
 * INTERFACE que define el CONTRATO de las 10 actividades solicitadas.
 *
 * Pilares de POO aplicados:
 * - ABSTRACCION : declara las operaciones sin implementarlas.
 * - POLIMORFISMO: cualquier clase que la implemente puede ser tratada
 *                  como un objeto de tipo OperacionesMatriz.
 *
 * @author Taipei Sulla Fernando
 */
public interface OperacionesMatriz {

    // Actividad 1
    int eliminarElemento(int x);

    // Actividad 2
    int modificarElemento(int x, int nuevoValor);

    // Actividad 3
    int sumarElementos();

    // Actividad 4
    int obtenerMenor();

    int obtenerMayor();

    double obtenerPromedio();

    void visualizarEstadisticas();

    // Actividad 5
    int sumaExtremosL();

    // Actividad 6
    int sumaExtremosJ();

    // Actividad 7
    int sumaExtremosU();

    // Actividad 8
    void visualizarMenorPorFila();

    // Actividad 9
    int contarPares();
}

```

```

// Actividad 10
void rotarFilasDerecha();
}

```

CMatrix.java — clase con la lógica de todas las operaciones

```

package clases;

/**
 * Clase CMatrix: representa una matriz cuadrada de numeros enteros y
 * todas las operaciones que se pueden realizar sobre ella.
 *
 * HERENCIA      : extends Matriz -> reutiliza atributos y metodos base.
 * POLIMORFISMO : implements OperacionesMatriz -> cumple el contrato de las 10 actividades.
 *
 * @author Taipei Sulla Fernando
 */
public class CMatrix extends Matriz implements OperacionesMatriz {

    // ===== Constructores (SOBRECARGA) =====

    /** Constructor por defecto: matriz de tamano 0. */
    public CMatrix() {
        super();
    }

    /** Constructor sobrecargado: recibe el tamano y crea el arreglo. */
    public CMatrix(int tamano) {
        super(tamano);
    }

    /** Constructor de copia: crea una nueva matriz a partir de otra existente. */
    public CMatrix(int[][] matrizOriginal) {
        super(matrizOriginal);
    }

    // =====
    // METODOS BASE (trabajo anterior)
    // =====

    public void leerNumero() {
        for (int i = 0; i < getTamano(); i++) {
            for (int j = 0; j < getTamano(); j++) {
                setElemento(i, j, leerEntero("Ingrese el elemento ["
                    + (i + 1) + "][" + (j + 1) + "] : "));
            }
        }
    }

    /** Implementacion OBLIGATORIA del metodo abstracto heredado de Matriz. */
    @Override
    public void escritura() {
        System.out.println("\tListado");
        System.out.println("\t=====");
        for (int i = 0; i < getTamano(); i++) {
            for (int j = 0; j < getTamano(); j++) {
                System.out.print(getElemento(i, j) + "\t");
            }
            System.out.println();
        }
    }
}

```

```

public void sumaFila() {
    for (int i = 0; i < getTamano(); i++) {
        int sumaF = 0;
        for (int j = 0; j < getTamano(); j++) {
            sumaF += getElemento(i, j);
        }
        System.out.println("La suma de la fila " + (i + 1) + " es: " + sumaF);
    }
}

public void sumaColumna() {
    for (int j = 0; j < getTamano(); j++) {
        int sumaCol = 0;
        for (int i = 0; i < getTamano(); i++) {
            sumaCol += getElemento(i, j);
        }
        System.out.println("La suma de la columna " + (j + 1) + " es: " + sumaCol);
    }
}

public void sumaDiagonalMayor() {
    int aux = 0;
    for (int i = 0; i < getTamano(); i++) {
        aux += getElemento(i, i);
    }
    System.out.println("La suma de la diagonal mayor es: " + aux);
}

public void visualizarElemento() {
    int valorBuscado, posicion = 0, cont = 1, fila = 0, columna = 0;
    boolean encontrado = false;

    valorBuscado = leerEntero("Ingrese elemento a buscar: ");

    for (int i = 0; i < getTamano(); i++) {
        for (int j = 0; j < getTamano(); j++) {
            if (getElemento(i, j) == valorBuscado) {
                posicion = cont;
                fila = i;
                columna = j;
                encontrado = true;
            }
            cont++;
        }
    }

    if (encontrado) {
        System.out.println("El elemento se encuentra en la posicion " + posicion);
        System.out.println("El elemento se encuentra en la fila " + (fila + 1) + ", columna "
+ (columna + 1));
    } else {
        System.out.println("El elemento no se encuentra en la matriz");
    }
}

// =====
//  ACTIVIDAD 1: Eliminar un elemento "x" de la matriz
// =====
/**
 * Elimina logicamente todas las apariciones del valor x.
 * En Java una matriz tiene tamano FIJO, por lo tanto no se puede quitar
 * una celda; la eliminacion se realiza asignando el valor neutro 0.
 */

```

```

    * @param x valor a eliminar
    * @return cantidad de elementos eliminados
    */
@Override
public int eliminarElemento(int x) {
    int contador = 0;
    for (int i = 0; i < getTamano(); i++) {
        for (int j = 0; j < getTamano(); j++) {
            if (getElemento(i, j) == x) {
                setElemento(i, j, 0);
                contador++;
                System.out.println("Elemento " + x + " eliminado de la fila "
                    + (i + 1) + ", columna " + (j + 1));
            }
        }
    }
    if (contador == 0) {
        System.out.println("El elemento " + x + " no existe en la matriz.");
    } else {
        System.out.println("Total de elementos eliminados: " + contador);
    }
    return contador;
}

// =====
// ACTIVIDAD 2: Modificar un elemento "x" de la matriz
// =====
/**
 * Reemplaza todas las apariciones del valor x por un nuevo valor.
 *
 * @param x valor actual que se desea cambiar
 * @param nuevoValor valor que se colocara en su lugar
 * @return cantidad de elementos modificados
 */
@Override
public int modificarElemento(int x, int nuevoValor) {
    int contador = 0;
    for (int i = 0; i < getTamano(); i++) {
        for (int j = 0; j < getTamano(); j++) {
            if (getElemento(i, j) == x) {
                setElemento(i, j, nuevoValor);
                contador++;
                System.out.println("Fila " + (i + 1) + ", columna " + (j + 1)
                    + ": " + x + " -> " + nuevoValor);
            }
        }
    }
    if (contador == 0) {
        System.out.println("El elemento " + x + " no existe en la matriz.");
    } else {
        System.out.println("Total de elementos modificados: " + contador);
    }
    return contador;
}

// =====
// ACTIVIDAD 3: Sumar todos los elementos de la matriz
// =====
@Override
public int sumarElementos() {
    int suma = 0;
    for (int i = 0; i < getTamano(); i++) {
        for (int j = 0; j < getTamano(); j++) {
            suma += getElemento(i, j);
        }
    }
}

```

```

    }
}
return suma;
}

/** Metodo de presentacion: muestra la suma total en pantalla. */
public void visualizarSumaTotal() {
    System.out.println("La suma de todos los elementos de la matriz es: " + sumarElementos());
}

// =====
// ACTIVIDAD 4: Menor, mayor y promedio
// =====
@Override
public int obtenerMenor() {
    int menor = getElemento(0, 0);
    for (int i = 0; i < getTamano(); i++) {
        for (int j = 0; j < getTamano(); j++) {
            if (getElemento(i, j) < menor) {
                menor = getElemento(i, j);
            }
        }
    }
    return menor;
}

@Override
public int obtenerMayor() {
    int mayor = getElemento(0, 0);
    for (int i = 0; i < getTamano(); i++) {
        for (int j = 0; j < getTamano(); j++) {
            if (getElemento(i, j) > mayor) {
                mayor = getElemento(i, j);
            }
        }
    }
    return mayor;
}

@Override
public double obtenerPromedio() {
    int cantidad = getTamano() * getTamano();
    return (double) sumarElementos() / cantidad;
}

@Override
public void visualizarEstadisticas() {
    System.out.println("El numero MENOR de la matriz es : " + obtenerMenor());
    System.out.println("El numero MAYOR de la matriz es : " + obtenerMayor());
    System.out.printf("El PROMEDIO de la matriz es      : %.2f%n", obtenerPromedio());
}

// =====
// ACTIVIDAD 5: Suma de los extremos que forman la letra L
// =====
/**
 * Forma de la L (ejemplo 4x4):
 *   X . . .
 *   X . . .
 *   X . . .
 *   X X X X
 * Primera columna completa + ultima fila (sin repetir la esquina).
 */
@Override

```

```

public int sumaExtremosL() {
    int n = getTamano();
    int suma = 0;
    for (int i = 0; i < n; i++) {           // palo vertical: primera columna
        suma += getElemento(i, 0);
    }
    for (int j = 1; j < n; j++) {           // base horizontal: ultima fila
        suma += getElemento(n - 1, j);
    }
    return suma;
}

public void visualizarSumaL() {
    System.out.println("La suma de los elementos que forman la letra L es: " +
sumaExtremosL());
}

// =====
// ACTIVIDAD 6: Suma de los extremos que forman la letra J
// =====
/**
 * Forma de la J (ejemplo 4x4):
 *   . . . X
 *   . . . X
 *   . . . X
 *   X X X X
 * Ultima columna completa + ultima fila (sin repetir la esquina).
 */
@Override
public int sumaExtremosJ() {
    int n = getTamano();
    int suma = 0;
    for (int i = 0; i < n; i++) {           // palo vertical: ultima columna
        suma += getElemento(i, n - 1);
    }
    for (int j = 0; j < n - 1; j++) {       // base horizontal: ultima fila
        suma += getElemento(n - 1, j);
    }
    return suma;
}

public void visualizarSumaJ() {
    System.out.println("La suma de los elementos que forman la letra J es: " +
sumaExtremosJ());
}

// =====
// ACTIVIDAD 7: Suma de los extremos que forman la letra U
// =====
/**
 * Forma de la U (ejemplo 4x4):
 *   X . . X
 *   X . . X
 *   X . . X
 *   X X X X
 * Primera columna + ultima columna (sin la ultima fila) + ultima fila completa.
 */
@Override
public int sumaExtremosU() {
    int n = getTamano();
    int suma = 0;
    for (int i = 0; i < n - 1; i++) {       // palo izquierdo
        suma += getElemento(i, 0);
    }

```

```

        for (int i = 0; i < n - 1; i++) {           // palo derecho
            suma += getElemento(i, n - 1);
        }
        for (int j = 0; j < n; j++) {               // base de la U
            suma += getElemento(n - 1, j);
        }
        return suma;
    }

    public void visualizarSumaU() {
        System.out.println("La suma de los elementos que forman la letra U es: " +
sumaExtremosU());
    }

    // =====
    // ACTIVIDAD 8: Menor numero de cada fila
    // =====
    @Override
    public void visualizarMenorPorFila() {
        for (int i = 0; i < getTamano(); i++) {
            int menorFila = getElemento(i, 0);
            int columna = 0;
            for (int j = 1; j < getTamano(); j++) {
                if (getElemento(i, j) < menorFila) {
                    menorFila = getElemento(i, j);
                    columna = j;
                }
            }
            System.out.println("El menor numero de la fila " + (i + 1) + " es: "
                + menorFila + " (columna " + (columna + 1) + ")");
        }
    }

    // =====
    // ACTIVIDAD 9: Contar los numeros pares de la matriz
    // =====
    @Override
    public int contarPares() {
        int contador = 0;
        for (int i = 0; i < getTamano(); i++) {
            for (int j = 0; j < getTamano(); j++) {
                if (getElemento(i, j) % 2 == 0) {
                    contador++;
                }
            }
        }
        return contador;
    }

    public void visualizarPares() {
        System.out.println("La cantidad de numeros PARES en la matriz es : " + contarPares());
        System.out.println("La cantidad de numeros IMPARES en la matriz es: "
            + (getTamano() * getTamano() - contarPares()));
    }

    // =====
    // ACTIVIDAD 10: Mover las filas una posicion a la derecha
    // =====
    /**
     * Desplaza cada fila una posicion a la derecha (rotacion circular):
     * el ultimo elemento de la fila pasa a ser el primero.
     */
    @Override
    public void rotarFilasDerecha() {

```

```

        int n = getTamano();
        for (int i = 0; i < n; i++) {
            int ultimo = getElemento(i, n - 1);    // 1) se guarda el ultimo
            for (int j = n - 1; j > 0; j--) {        // 2) se desplaza de derecha a izquierda
                setElemento(i, j, getElemento(i, j - 1));
            }
            setElemento(i, 0, ultimo);              // 3) el ultimo pasa al inicio
        }
        System.out.println("Filas desplazadas una posicion a la derecha.");
    }

    // =====
    //  UTILITARIO: carga una matriz de ejemplo (para pruebas / evidencias)
    //  =====
    public void cargarEjemplo() {
        int[][] demo = {
            {8, 3, 12, 5},
            {7, 20, 1, 9},
            {4, 15, 6, 11},
            {2, 10, 14, 13}
        };
        setMatriz(demo);
        System.out.println("Matriz de ejemplo 4x4 cargada correctamente.");
    }

    // =====
    //  SOBRESCRITURA DE METODOS DE Object (POLIMORFISMO)
    //  =====

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < getTamano(); i++) {
            for (int j = 0; j < getTamano(); j++) {
                sb.append(getElemento(i, j)).append("\t");
            }
            sb.append("\n");
        }
        return sb.toString();
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (!(obj instanceof CMatrix)) {
            return false;
        }
        CMatrix otra = (CMatrix) obj;
        if (this.getTamano() != otra.getTamano()) {
            return false;
        }
        for (int i = 0; i < getTamano(); i++) {
            for (int j = 0; j < getTamano(); j++) {
                if (this.getElemento(i, j) != otra.getElemento(i, j)) {
                    return false;
                }
            }
        }
        return true;
    }

    @Override

```

```

    public int hashCode() {
        int hash = 7;
        hash = 31 * hash + getTamano();
        for (int i = 0; i < getTamano(); i++) {
            for (int j = 0; j < getTamano(); j++) {
                hash = 31 * hash + getElemento(i, j);
            }
        }
        return hash;
    }
}

```

AplicaMatriz.java — clase ejecutable con el menú de opciones

```

package clases;

/**
 * Clase PRINCIPAL (capa de presentacion).
 * Solo se encarga de mostrar el menu y pedir datos al usuario;
 * toda la logica esta dentro del objeto CMatrix.
 *
 * Esto demuestra el principio de RESPONSABILIDAD UNICA de la POO.
 *
 * @author Taipe Sulla Fernando
 */
public class AplicaMatriz {

    // Instancia del objeto (constructor por defecto)
    static CMatrix objArreglo = new CMatrix();

    public static void main(String[] args) {
        menu();
    }

    static void menu() {
        int opcion, tam, x, nuevo;

        do {
            System.out.println("\n\n          MENU DE OPCIONES - MATRICES");
            System.out.println("=====");
            System.out.println("  CONFIGURACION");
            System.out.println("  1.- Crear el Arreglo (matriz n x n)");
            System.out.println("  2.- Ingresar Datos");
            System.out.println("  3.- Mostrar Datos");
            System.out.println("-----");
            System.out.println("  OPERACIONES BASE");
            System.out.println("  4.- Sumar Filas");
            System.out.println("  5.- Sumar Columnas");
            System.out.println("  6.- Sumar Diagonal Mayor");
            System.out.println("  7.- Visualizar el contenido de una posicion");
            System.out.println("-----");
            System.out.println("  ACTIVIDADES DESARROLLADAS");
            System.out.println("  8.- Eliminar un elemento X");
            System.out.println("  9.- Modificar un elemento X");
            System.out.println(" 10.- Sumar todos los elementos");
            System.out.println(" 11.- Menor, Mayor y Promedio");
            System.out.println(" 12.- Suma de extremos en forma de L");
            System.out.println(" 13.- Suma de extremos en forma de J");
            System.out.println(" 14.- Suma de extremos en forma de U");
            System.out.println(" 15.- Menor numero de cada fila");
            System.out.println(" 16.- Contar numeros pares");
            System.out.println(" 17.- Mover filas una posicion a la derecha");

```

```

System.out.println("-----");
System.out.println(" 18.- Cargar matriz de ejemplo (4x4)");
System.out.println(" 0.- Salir");
System.out.println("=====");

opcion = Matriz.leerEntero("Ingrese una alternativa: ");
System.out.println();

switch (opcion) {
    case 1:
        tam = Matriz.leerEntero("Ingrese tamano del Arreglo: ");
        objArreglo.setTamano(tam); // el setter crea el arreglo
        System.out.println("Matriz de " + tam + " x " + tam + " creada.");
        break;

    case 2:
        if (objArreglo.validarMatriz()) {
            objArreglo.leerNumero();
        }
        break;

    case 3:
        if (objArreglo.validarMatriz()) {
            objArreglo.escritura();
        }
        break;

    case 4:
        if (objArreglo.validarMatriz()) {
            objArreglo.sumaFila();
        }
        break;

    case 5:
        if (objArreglo.validarMatriz()) {
            objArreglo.sumaColumna();
        }
        break;

    case 6:
        if (objArreglo.validarMatriz()) {
            objArreglo.sumaDiagonalMayor();
        }
        break;

    case 7:
        if (objArreglo.validarMatriz()) {
            objArreglo.visualizarElemento();
        }
        break;

    // ===== ACTIVIDAD 1 =====
    case 8:
        if (objArreglo.validarMatriz()) {
            x = Matriz.leerEntero("Ingrese el elemento X a eliminar: ");
            objArreglo.eliminarElemento(x);
            System.out.println("\nMatriz resultante:");
            objArreglo.escritura();
        }
        break;

    // ===== ACTIVIDAD 2 =====
    case 9:
        if (objArreglo.validarMatriz()) {

```

```

        x = Matriz.leerEntero("Ingrese el elemento X a modificar: ");
        nuevo = Matriz.leerEntero("Ingrese el nuevo valor: ");
        objArreglo.modificarElemento(x, nuevo);
        System.out.println("\nMatriz resultante:");
        objArreglo.escritura();
    }
    break;

// ===== ACTIVIDAD 3 =====
case 10:
    if (objArreglo.validarMatriz()) {
        objArreglo.visualizarSumaTotal();
    }
    break;

// ===== ACTIVIDAD 4 =====
case 11:
    if (objArreglo.validarMatriz()) {
        objArreglo.visualizarEstadisticas();
    }
    break;

// ===== ACTIVIDAD 5 =====
case 12:
    if (objArreglo.validarMatriz()) {
        objArreglo.visualizarSumaL();
    }
    break;

// ===== ACTIVIDAD 6 =====
case 13:
    if (objArreglo.validarMatriz()) {
        objArreglo.visualizarSumaJ();
    }
    break;

// ===== ACTIVIDAD 7 =====
case 14:
    if (objArreglo.validarMatriz()) {
        objArreglo.visualizarSumaU();
    }
    break;

// ===== ACTIVIDAD 8 =====
case 15:
    if (objArreglo.validarMatriz()) {
        objArreglo.visualizarMenorPorFila();
    }
    break;

// ===== ACTIVIDAD 9 =====
case 16:
    if (objArreglo.validarMatriz()) {
        objArreglo.visualizarPares();
    }
    break;

// ===== ACTIVIDAD 10 =====
case 17:
    if (objArreglo.validarMatriz()) {
        System.out.println("Matriz ORIGINAL:");
        objArreglo.escritura();
        objArreglo.rotarFilasDerecha();
        System.out.println("\nMatriz DESPLAZADA:");
    }

```

```
        objArreglo.escritura();
    }
    break;

    case 18:
        objArreglo.cargarEjemplo();
        objArreglo.escritura();
        break;

    case 0:
        System.out.println("Gracias por usar el programa. Hasta pronto!");
        break;

    default:
        System.out.println("Opcion no valida, ingrese otra opcion.");
    }

    } while (opcion != 0);
}
}
```